

METHODS — כל כלי סטטיסטי בפרויקט, בעברית

פשוטה

עודכן: סוף יום 7 · 17.8.2026 **למה זה קיים:** יש בפרויקט הרבה מבחנים, וקל לאבד את החוט. המסמך הזה עונה על שלוש שאלות לכל כלי — **מה הוא שואל · למה הוא ולא אחר · איפה הוא בקוד**.
לא תיאוריה. רק מה שאנחנו משתמשים בו בפועל.

0. מפה מהירה

כלי	מה הוא עונה	איפה
OLS / רגרסיה	כמה X מסביר את Y	<code>cost_model.py</code>
טרנספורמציות log	להפוך יחסים כפליים לחיבוריים	<code>cost_model.py</code>
WLS (משוקלל)	רגרסיה כשלא כל תצפית שווה	<code>production_model.py</code>
GLM בינומי	רגרסיה כש-Y הוא שיעור בין 0 ל-1	<code>avaliabillity_model.py</code>
התכווצות	לרסן אומדנים ממדגם קטן	<code>roster_optimizer.py</code>
כיול	האם התחזיות ברמה הנכונה	הכל
OOS	האם זה עובד על מה שלא ראינו	<code>Oos_hapoel.py</code>
מונטה קרלו	להריץ עונה 400 פעם	<code>roster_optimizer.simulate</code>
פיזור-יתר / בטא-בינומי	האם ה שיעור עצמו לא ידוע	<code>avail_uncertainty.py</code>
תכנון לינארי בשלמים (PuLP)	לבחור סגל תחת אילוצים	<code>optimise_consistent.py</code>
סווייפ רגישות	כמה התשובה תלויה בהנחה	<code>depth_sweep.py</code>
תחזית נעולה מראש	למנוע התאמה בדיעבד	<code>Oos_hapoel.py</code> ואילך

1. מה שכבר הכרת

רגרסיה (OLS)

מה שואלים: בהינתן PIR של אשתקד, מה השכר הצפוי. **מה שקוראים:** $\log(\text{cost}) = \beta_0 + \beta_1 \cdot \text{PIR} + \dots$
 β_1 הוא הכיוון והעוצמה. p הוא הסיכוי לקבל תוצאה כזו במקרה — קטן מ-0.05 נחשב "לא מקרי". R^2 הוא כמה מהשונות הוסברה.

אצלנו: $\beta_1 = 0.252$, $p = 0.016$ — PIR מסביר שכר. $R^2 = 0.953$ על מכבי, 0.709 על הפועל.

טרנספורמציות log

למה: שכר גדל **כפולית** — מ-300K ל-600K זה "פי 2", כמו מ-3M ל-6M. רגרסיה רגילה מניחה תוספת קבועה. `log` הופכת כפל לחיבור, ואז הרגרסיה מתאימה. **מחיר:** כשחוזרים ל-`exp` מקבלים **חציון**, לא ממוצע. לכן יש `smear` בקוד — תיקון שמחזיר אותנו לממוצע.

משתנה קטגוריאלי

`el_seasons` — מספר עונות ביורוליג. אפשר להתייחס אליו כמספר רציף (כל עונה מוסיפה אותו דבר) או כקטגוריות (עונה 1, 2, 3, +4 נפרדות). **המחלוקת הפתוחה:** הקוד רץ רציף; המפרט הרשמי אומר קטגוריאלי. עדיין לא הוכרע.

סטיית תקן

הפיזור סביב הממוצע. **החשוב:** אנחנו מבחינים בין **שני סוגים** —

	מה זה	דוגמה
שארית	הפיזור האמיתי בין שחקנים	שחקנים באמת שונים זה מזה
שגיאת אמידה	כמה אנחנו לא בטוחים בתחזית	מדגם קטן → לא בטוחים

זה היה באג ביום 6: ענשנו שחקנים לפי **השארית** — כלומר הענשנו שחקן שבאמת טוב יותר מהתחזית. אבל **הוא מה שמחפשים**. קללת המנצח נובעת משגיאת **האמידה**. התיקון: מעבר ל-`se_mean`.

2. מה שהוספנו

WLS — רגרסיה משוקללת

הבעיה: שחקן ששיחק 34 משחקים ושחקן ששיחק 3 — ה-`ppm` של הראשון אמין הרבה יותר. **הפתרון:** לתת לכל תצפית **משקל**. אצלנו המשקל הוא סך הדקות. **איפה:** `production_model.py`, `sm.WLS(..., weights=minutes_tot)`

GLM בינומי — כש-Y הוא שיעור

הבעיה: זמינות היא `משחקים / משחקים אפשריים` — תמיד בין 0 ל-1. רגרסיה רגילה יכולה לנבא 1.3 או -0.2, וזה חסר משמעות. **הפתרון:** מודל שמובנה כך שהתוצאה לא יכולה לצאת מהתחום. הקשר הוא `logit`:

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

הצד השמאלי חופשי לנוע מ- $-\infty$ ל- $+\infty$, והצד הימני תמיד בין 0 ל-1.

אצלנו: דקות_קודם + 1.456 * זמינות_קודם - 0.008 * גיל - 0.962 = `logit(frac)` **איפה:** `avaliability_model.py`

התכווצות (Shrinkage)

הבעיה: שחקן ששיחק 3 משחקים עם PIR 20 — האם הוא באמת כזה, או שהיה לו מזל? **הפתרון:** למשוך את האומדן שלו לכיוון ממוצע הליגה, **ככל שיש עליו פחות דאטה**.

$$w = \frac{k}{k + \text{משחקים}} \quad k = 40$$

$$\text{ממוצע_הליגה} \cdot (1-w) + \text{שלו} \cdot w = \text{אומדן}$$

3 משחקים $\rightarrow w = 0.07$, כלומר 93% ממוצע ליגה. 34 משחקים $\rightarrow w = 0.46$. זה פרמטר מוצהר, ו- k נסרק ב-20 ו-60 כבדיקת רגישות.

כיול – ולמה הוא חשוב יותר מדירוג

דירוג: האם סידרנו את השחקנים נכון. **כיול:** האם הרמה נכונה.

מודל שאומר "כולם יהיו זמינים 90%" כשבפועל זה 75% — **מדרג מושלם ומשקר על הרמה**. וכשמכניסים אותו לאופטימיזר שמחלק תקציב, כל ההקצאה שגויה.

איך בודקים: מחלקים את השחקנים לעשירונים לפי התחזית, ובכל עשירון משווים תחזית ממוצעת מול מה שקרה. **הפער המרבי** הוא הציון.

אצלנו: זמינות — פער 0.054 מול סף 0.070 שנרשם מראש ✓ · תפוקה — 0.096 מול 0.05 $\times$ **נכשל**

OOS — מחוץ למדגם

הרעיון: לאמן על 2023 ומטה, לנבא את 2024, ולהשוות למה שקרה. מודל שנראה מצוין על הדאטה שלו ונופל מחוץ לו — **התאים את עצמו לרעש**.

אצלנו: זמינות — 0.753 חזוי מול 0.740 בפועל. פער $+0.013$ ✓

מונטה קרלו

הרעיון: במקום לחשב "מה יקרה בממוצע", להריץ את העונה 400 פעם עם הגרלות ולראות את **ההתפלגות**.

מה מוגרל אצלנו, ובאיזה סדר:

השחקן הוא מה שהוא — (תחזית, סטיית_תקן) $\sim N(\mu, \sigma)$: פעם אחת לעונה
בכל משחק : נוכח / לא נוכח

הסדר הזה הוא כל העניין. אם הכוכב מתגלה חלש מהצפוי, זה נכון **לכל העונה**, וצריך למי להעביר את הדקות. זה מה שנותן לעומק ערך, ומה שהחישוב הדטרמיניסטי לא רואה.

פיזור-יתר ובטא-בינומי

השאלה: ה-GLM נותן "לשחקן הזה 78% זמינות". אבל **כמה אנחנו בטוחים במספר הזה עצמו?**

המבחן: אם המודל מושלם, הסטייה של כל תצפית מהתחזית צריכה להיות בגודל שהמודל הבינומי צופה. מחשבים

ממוצע $\phi = [\text{בפועל} - \text{חזוי}]^2 / \text{השונות_הצפויה}$

$\phi = 1 \rightarrow$ אין אי-ודאות נוספת. $\phi = 10.38$ (מה שקיבלנו) $\rightarrow$ **השונות גדולה פי עשרה ממה שהמודל מניח.**

התיקון — בטא-בינומי: קודם מגרילים את השיעור האמיתי של השחקן מהתפלגות בטא סביב התחזית, ורק אז מגרילים משחק-משחק.

אצלנו: ס"ת של השיעור האמיתי סביב 0.78 הוא **0.228** — שחקן שנחזה 78% הוא בפועל בטווח 55%-100%.

איפה: `avail_uncertainty.py`

תכנון לינארי בשלמים (PuLP)

מה זה: בחירה תחת אילוצים. משתני החלטה בינאריים (x_i = בסגל או לא) ורציפים (e_i = דקות), ומקסמים ביטוי לינארי.

```
max  Σ ppm(i)·e(i)
      Σ e(i) ≤ 200          ·  e(i) ≤ 32·זמיןות(i)·x(i)
      Σ עלות(i)·x(i) ≤ תקציב
      12 ≤ Σ x(i) ≤ 16      ·  רצפות ותקרות עמדה
```

הכלל הקריטי שנלמד ביום 7: הפונקציה שממקסמים חייבת להיות בדיוק הפונקציה שמנקדת. אם לא — הידוק אילוף יכול להעלות את הערך, וזה בלתי אפשרי מתמטית. זו הייתה ההוכחה לבאג.

סווייפ רגישות

מתי: כשמספר בקוד הוא הנחה ולא מדידה. מה עושים: מריצים על פני טווח שלם ומדווחים את הטווח, לא ערך אחד. דוגמה חיה: רמת המחלף. מדדנו 0.127, אבל היא נעה בין 0.030 ל-0.179 לפי הסף שבוחרים. לכן היא נסרקת ולא נבחרת.

תחזית נעולה מראש

לא כלי סטטיסטי — כלל עבודה, והוא החשוב ביותר בפרויקט. לפני שמריצים, כותבים בראש הקובץ מה אנחנו צופים ולמה. אחר כך אי אפשר להתאים את הפרשנות לתוצאה. זה עבד: ביום 6 חזיתי שאצל הפועל התקציב לא מגביל. ניצול היה 99.7%. התחזית נפלה, וזה היה הממצא. וזה נשמט: בבוקר יום 7 הרצתי שלושה קבצים בלי אף תחזית. הבוקר הזה הוא היחיד שכל מסקנותיו נפלו.

3. הכשלים החוזרים — מה לבדוק לפני שמדווחים מספר

הכשל	דוגמה בפרויקט
שני צדי ההשוואה אינם אותם אנשים	בנצ'מרק מכבי — 9 שחקנים במקום 12, בלי שני הטובים
סקלה מעונה אחרת	תמחור מועמדי 2024 לפי שכר ממוצע של 2023
מכנה לא נבדק	ליוד — 22.8 דק' למשחק אחרי משחק אחד
מזהה נחשב למספר	Player_ID , player_code , gamecode — שלוש פעמים
מבחן שלא יכול להראות אחרת	בחירת גודל סגל לפי ניקוד שמונטוני יורד
כישלון שקט	Skip and continue · .gitignore שחוסם דאטה
סינון חסר	בוקסקור כולל פלייאוף, player_season לא
התניה חסרה	מיצוע דקות כולל משחקים שבהם השחקן נעדר

ארבעה מתוך שמונה הם אותה משפחה: דיווחתי מספר לפני שווידאתי ששני הצדדים עומדים על אותו בסיס.

4. איפה כל מבחן יושב

קובץ	מה הוא בודק
cost_model.py	OLS על $\log(\text{share})$ — כמה עולה שחקן
production_model.py	WLS — ppm צפוי
avaliabillity_model.py	GLM בינומי — שיעור משחקים
audits/survival_model.py	שרידות (הוחלף, לא נזרק)
0os_hapoel.py	מבחן חוץ-מדגמי עם תחזיות נעולות
league_jump_test.py	הליגה הישראלית — הטיית בחירה
roster_optimizer.py	PuLP + מונטה קרלו
optimise_consistent.py	● יישור המטרה לניקוד
optimizer_backtest.py	הבקטסט של המנוע
backtest_diagnostics.py	שלוש בדיקות על המבחן
roster_membership_audit.py	מי נחשב לסגל
benchmark_matched.py	ההשוואה מול המועדון
curse_decomp.py	פירוק קללת המנצח
replacement_level.py	מדידת רמת המחליף
depth_sweep.py	סווייפ רגישות
split_multiclub.py	פיצול לפי מועדון מבוקסקור
avail_uncertainty.py	פיזור-יתר בזמינות
curse_redistribution.py	קללה — ממצא או ארטיפקט
absence_position_test.py	לאן דקות הנעדר הולכות
final_day7.py	ההרצה המסכמת